

Technologies for IoT Ecosystems

Software Lab

INTRODUCTION

This laboratory course guides you through the design and implementation of a complete Smart Home IoT ecosystem built entirely in Python and, in the final part, extended to physical hardware (i.e. the Arduino). Across thirteen progressive exercises (split in three parts), you will construct every layer of a real-world IoT platform: from the REST endpoints that expose individual sensor readings (Software Lab – Part 1), through the centralized Catalog that ties the platform together (Software Lab – Part 2), up to a fully distributed controller that reacts autonomously to physical sensor data and drives actuators on an Arduino board (Software Lab – Part 3).

This is not a collection of isolated exercises. Each component you build is a functional piece of a larger system. This explicitly requires you to run multiple services simultaneously and to distribute the IoT ecosystem across several computer systems. By the final session all thirteen components will cooperate over both REST and MQTT, with a physical Arduino as a first-class citizen of the ecosystem.

LEARNING OBJECTIVES

By the end of this lab you will be able to:

- Apply Object-Oriented Programming principles throughout.
- Design and implement RESTful web services in Python using the CherryPy framework and its MethodDispatcher, selecting the correct HTTP verb (GET, POST, PUT, DELETE) for each operation.
- Format all inter-service data exchange using SenML (Sensor Markup Language), the standard lightweight data format used in constrained IoT environments.

TECHNICAL PREREQUISITES

Before starting, ensure the following are installed in your Python environment:

```
pip install cherrypy requests
```

Exercise 01 — Smart Home Sensor Reader

Develop in OOP a CherryPy RESTful web service that exposes temperature, humidity, and motion sensor readings for a simulated smart home. Define the right HTTP method (among GET, POST, PUT and DELETE) with query parameters to select the room and the sensor type. All responses must use SenML (Sensor Markup Language) as data format. Handle errors for unknown rooms or sensor types with appropriate HTTP status codes.

Requirements	1. Define a SmartHomeSensorService class with exposed=True and a GET method. 2. Simulate at least three rooms (living_room, kitchen, bedroom) each with temperature (°C), humidity (%RH), and motion (boolean) sensors. 3. Return HTTP 404 if the requested room does not exist. 4. Return HTTP 400 if the requested sensor type is unknown. 5. All payloads must be valid SenML array of events (e): include base-name (bn), base-time (bt), name (n), unit (u), and value (v).
Output / JSON	HTTP 200 — SenML JSON array with base-name, base-time, name, unit, value HTTP 400 — {"error": "unknown sensor type", "valid_types": [...]} HTTP 404 — {"error": "room not found", "available_rooms": [...]}
Tips & Notes	⚠ Use random.uniform() to simulate sensor readings within realistic ranges. ⚠ Use time.time() for base-time (bt) — seconds since Unix epoch. Test all endpoints with curl or Postman.

Exercise 02 — Smart Home Sensor Reader — URI-Style

Extend the RESTful sensor service from Exercise 01 to expose full URL-path (slash-separated) endpoints. Parameters must be embedded in the URI rather than passed as query parameters. This exercise reinforces URI resource modelling and hierarchical path design.

Endpoints	GET /sensors/<room>/<type> — single sensor reading GET /sensors/<room> — all sensors in a room GET /sensors — all rooms and sensors
Requirements	1. Reuse the sensor simulation logic from Exercise 01 inside the same class or via composition. 2. Parse the URI to extract room and type path segments. 3. Return HTTP 404 for unknown rooms, HTTP 400 for unknown sensor types. 4. The SenML base-name must embed the room name to allow unambiguous identification. 5. Test all endpoints with curl or Postman.
Output / JSON	Same SenML format as Exercise 01 — base-name now includes room identifier.
Tips & Notes	⚠ Use random.uniform() to simulate sensor readings within realistic ranges.

⚠ Use `time.time()` for base-time (bt) — seconds since Unix epoch.

Test all endpoints with curl or Postman.

Exercise 03 — Smart Home Actuator Control

Extend the smart home service with actuator control capabilities. Define the right HTTP method (among GET, POST, PUT and DELETE) to send commands to actuators (thermostat, lights, blinds) in each room. Then, define the right HTTP method to read the current actuator state. The command payload and the state response must both use SenML format. All state changes are stored in memory and reflected in subsequent HTTP calls.

Requirements

1. Model at least: thermostat (setpoint in °C), lights (on/off boolean), blinds (position 0–100%).
2. Return HTTP 422 (Unprocessable Entity) if the SenML body is malformed.
3. Return HTTP 404 for unknown room or device.
4. Store actuator states in a nested dict. Choose the right data-structure.

Tips & Notes

⚠ A thermostat setpoint below 10°C or above 30°C should return HTTP 400 (out of range).

Test all endpoints with curl or Postman.

Exercise 04 — Smart Home Event Log

Add an event logging subsystem to the smart home platform. Every sensor reading request and every actuator command must produce a timestamped log entry stored in memory. The log can be queried via REST with optional filtering by room, device type, or time window. Log entries must be formatted as SenML arrays. This service integrates with Exercises 01–03 by consuming and re-exposing their data.

Endpoints

POST	/log — append a SenML event record
GET	/log — retrieve all log entries
GET	/log?room=<r>&since=<epoch> — filter by room and/or time
GET	/log/<room> — log entries for a room
DELETE	/log?before=<epoch> — purge entries older than epoch

Requirements

1. Define an EventLog class that maintains an in-memory list of SenML records.
2. Choose the right HTTP Method for:
 - Appending (i.e., saving) one or more SenML records sent from the webclient to the webserver; assign a server-side log ID and an EPOCH timestamp.
 - Reading the full log as a JSON array of SenML array of events.
 - Supporting query parameters: room (match bn prefix), since (bt >= value), before (bt < value)
 - Removing all entries with bt < epoch and returning a count of deleted entries.

3. This service is used by Exercises 01–03: each sensor read / actuator command must save a log entry. Update Exercises 01–03 accordingly if needed.

Tips & Notes

⚠ Use `time.time()` to generate the server-side timestamp.

Test all endpoints with curl or Postman.